

Computer Networks (CSAI321)

Contest Exam Notes — Lectures 1 to 6

Concepts, Comparisons, Calculations, Protocol Flows & Scenarios

Compiled Study Notes

How to use this document. This is a single, self-contained revision resource built directly from Lectures 1–6 and mapped onto the official contest syllabus. Every section opens with the syllabus’s “Core Exam Areas” for that lecture, followed by full concept coverage, worked examples, comparison tables, analogies (as used in lecture), and an **EXAM FOCUS** box of the highest-yield facts. A consolidated **Master Formula Sheet** and **Rapid-Fire MCQ Recall Sheet** appear at the end. **Note:** Queuing-delay mathematics is explicitly out of scope (only L/R and d/s plug-in calculations are examinable).

Contents

1	Lecture 1: How the Internet Works	4
1.1	The Cloud Is Physical, Not Magic	4
1.2	Cloud Computing Providers	4
1.3	Networks Are Not New: A Brief History	4
1.4	Types of Networks by Geographical Scope	4
1.5	Physical Infrastructure of the Internet: Submarine Cables	4
1.6	The End-to-End Request Journey	5
1.7	Course Objective and Approach	5
1.8	Networking and GPUs (AI Infrastructure Preview)	5
1.9	Course Structure Preview	5
1.10	Grading Breakdown	5
1.11	Reference Textbooks	6
2	Lecture 2: Moving Data Through the Core	7
2.1	Circuit Switching vs. Packet Switching	7
2.1.1	Circuit Switching – “The Road Not Taken”	7
2.1.2	Packet Switching	7
2.2	Circuit Switching vs. Packet Switching – Comparison Table	7
2.3	Statistical Multiplexing	8
2.4	Sources of Delay – The Toll-Booth Caravan Analogy	8
2.5	Transmission vs. Propagation Delay – Toll-Booth Mapping	9
2.6	Worked Example: Transmission Delay	9
2.7	Worked Example: Propagation Delay – The Satellite Case	9
2.8	Throughput and Bottlenecks	9
2.9	Bridge to Lecture 3	10
3	Lecture 3: OSI and TCP/IP Models (Protocol Layers)	11

3.1	Why Layering?	11
3.1.1	Recap Bridge from Lecture 2	11
3.2	The Letter Analogy	11
3.3	What Layering Gives Us	11
3.4	Encapsulation and Decapsulation	12
3.5	The OSI Model	12
3.5.1	OSI Memory Tricks	12
3.6	The Seven OSI Layers – Full Detail	12
3.6.1	Layer 7 – Application	12
3.6.2	Layer 6 – Presentation	13
3.6.3	Layer 5 – Session	13
3.6.4	Layer 4 – Transport	13
3.6.5	Layer 3 – Network	13
3.6.6	Layer 2 – Data Link	13
3.6.7	Layer 1 – Physical	14
3.7	Layer 2 vs. Layer 3 – The Most Confused Distinction	14
3.8	Seven OSI Layers at a Glance	14
3.9	The TCP/IP Model	14
3.10	OSI vs. TCP/IP – Comparison Table	15
4	Lecture 4: Networking Devices, Topologies & the Cloud Network	16
4.1	Why Do We Need Network Devices?	16
4.1.1	The Scaling Problem	16
4.1.2	The Solution – The Pivot	16
4.2	Core Networking Devices	16
4.2.1	Hub – The 1980s Answer (Layer 1)	16
4.2.2	Switch – The 1990s Answer (Layer 2)	17
4.2.3	Hub vs. Switch – Comparison Table	17
4.2.4	Router – Reaching Another Network (Layer 3)	17
4.2.5	Firewall	17
4.2.6	Load Balancer	18
4.3	Devices by OSI Layer – The Single Most Important Table	18
4.4	Network Topologies	18
4.4.1	Star Topology	18
4.4.2	Mesh Topology	18
4.4.3	Hybrid Topology	19
4.4.4	Choosing a Topology by Environment	19
4.4.5	Fault Tolerance & Scalability	19
4.5	Cloud Networking – Hardware Meets Software	19
4.5.1	AWS Foundational Concepts	19
5	Lecture 5: HTTP, P2P & REST (Application Layer)	21
5.1	Network Architectures	21
5.1.1	Client-Server Architecture	21
5.1.2	Peer-to-Peer (P2P) Architecture	21
5.2	How Systems Communicate	21
5.3	The Application Layer (Layer 7)	21
5.4	HTTP – The Web’s Language	21
5.4.1	HTTP Request Structure	21
5.4.2	Request Line – Three Parts	22
5.4.3	HTTP Headers (Request)	22
5.4.4	HTTP Response Structure	22
5.4.5	HTTP Status Line – Three Parts	22
5.4.6	HTTP Response Headers	22

5.5	HTTP Status Codes	22
5.6	Statelessness and Cookies	23
5.7	Evolution of HTTP	23
5.7.1	HTTP/1.0 – 1996	23
5.7.2	HTTP/1.1 – 1997	23
5.7.3	Head-of-Line (HoL) Blocking	23
5.7.4	HTTP/1.1 Pipelining	23
5.7.5	HTTP/2 – 2015	24
5.7.6	HTTP/3 + QUIC – June 2022	24
5.8	HTTP/1.1 vs. HTTP/2 vs. HTTP/3 – Master Comparison	24
5.8.1	The Head-of-Line Blocking Evolution – Deeper View	24
5.9	REST APIs	25
5.9.1	HTTP Methods ↔ CRUD Mapping	25
5.9.2	REST Request/Response Flow	25
5.10	OpenAPI	25
6	Lecture 6: HTTPS and the TLS Handshake	26
6.1	The Padlock Question	26
6.2	Why HTTP Is Insecure	26
6.3	HTTPS = HTTP + TLS	26
6.4	Cryptography Fundamentals	26
6.4.1	Three Core Goals of Cryptography	26
6.5	Symmetric vs. Asymmetric Encryption	27
6.5.1	Symmetric Encryption	27
6.5.2	Asymmetric Encryption	27
6.6	The Key Insight of TLS	27
6.7	Trusting a Server: Digital Certificates	27
6.7.1	What Is a Certificate?	27
6.8	The TLS Handshake – Full Flow	28
6.9	Certificate Authority (CA) and Chain of Trust	28
6.10	Real-World Deployment: AWS	28
6.11	Symmetric vs. Asymmetric – Master Comparison	28
7	Master Formula Sheet (Lecture 2 Scope)	30
8	Rapid-Fire MCQ Recall Sheet	30
9	Cross-Lecture Concept Map (Scenario-Based Prep)	32

1 Lecture 1: How the Internet Works

EXAM FOCUS

Syllabus core areas: Internet as a network of networks; edge vs. core; access networks; end-to-end request journey; latency intuition; AWS Region/AZ/Edge vocabulary.

1.1 The Cloud Is Physical, Not Magic

The lecture opens with the provocative question: *“Is the cloud just someone else’s computer?”* The answer builds progressively: it is not one computer but **a lot of computers** arranged in massive data centers (example used: Google’s data-center infrastructure in Council Bluffs, Iowa). The core takeaway is that despite the abstract name “the cloud,” it is backed by real, physical infrastructure, and **all these servers must talk to each other** – which is the entire reason computer networks exist.

1.2 Cloud Computing Providers

Three major cloud service providers are named:

- **Amazon Web Services (AWS)**
- **Microsoft Azure**
- **Google Cloud Platform (GCP)**

These providers operate super-fast computer networks whose purpose is to efficiently deliver data (“0s and 1s”) to users at scale.

1.3 Networks Are Not New: A Brief History

EXAM FOCUS

In the **1960s**, a UCLA computer attempted to send the word **“login”** to a computer at the **Stanford Research Institute (SRI)**. Only the letters **“lo”** got through before the system crashed. This is the classic origin story of the Internet/ARPANET and a common trivia MCQ.

1.4 Types of Networks by Geographical Scope

Acronym	Full Form	Description
WAN	Wide Area Network	Covers large geographical areas – connects branch offices or is the Internet itself
MAN	Metropolitan Area Network	Spans a city; used by municipalities/large organizations
LAN	Local Area Network	Homes/offices; connects devices within a limited area
PAN	Personal Area Network	Very short range, e.g., phone ↔ smart-watch/headphones

EXAM FOCUS

Memorize scope ordering from smallest to largest: **PAN → LAN → MAN → WAN**. The Internet itself is best classified as a WAN (a “network of networks”).

1.5 Physical Infrastructure of the Internet: Submarine Cables

The Internet depends on a global physical infrastructure – most importantly, undersea (submarine) cables.

- The lecture states there were **597 cable systems in 2025**.
- These cables are physically installed and protected under the ocean.
- **Fishing anchors / trawlers** are highlighted as a **major cause of transatlantic cable/network traffic damage**.
- **Sharks** are presented (semi-humorously but factually referenced in networking lore) as another unusual physical threat to submarine cables.

EXAM FOCUS

If asked “what commonly damages submarine cables?” – the two lecture-highlighted causes are **fishing anchors/trawlers** (most significant, especially for transatlantic links) and **sharks** (secondary, illustrative threat).

1.6 The End-to-End Request Journey

Two scenarios illustrate why understanding the full journey of data matters:

1. **Instagram tap:** A user taps the Instagram icon and a reel plays in under a second. Most people mentally simplify the journey to just *WiFi* → *Instagram*, but the actual path between user and application involves many intermediate networking components and layers. Understanding this gap is a core purpose of the entire course.
2. **Netflix buffering at 10 PM:** When millions of concurrent users cause Netflix to buffer or fail, the course asks students to reason about what’s happening “behind the scenes” – a preview of congestion, bottlenecks, and infrastructure scale (developed further in Lecture 2 and Lecture 4).

1.7 Course Objective and Approach

- **Objective:** Demystify how networks work, because networks are everywhere.
- **Approach: Top-Down Approach** – start from applications (Web, HTTP, DNS, streaming – things students already use) and drill down through:

Application → Transport (TCP/UDP) → Network (IP/Routing) → Link → Physical

- **Philosophy:** Constantly answer “why” before “what.”

1.8 Networking and GPUs (AI Infrastructure Preview)

GPUs are connected using very fast network cards; networking is described as the **backbone of GPU clusters**. Specific figures mentioned: **200 Gb/s NIC** and **200 Gbps InfiniBand** as examples of high-speed network fabric connecting GPU clusters for AI training.

1.9 Course Structure Preview

- **Foundation:** how computing devices talk to each other; how data travels.
- **Hands-on Labs:** configure your own network.
- **Network Security:** introduction and application of security concepts.

1.10 Grading Breakdown

Component	Weight
Contests	20%
Projects	20%
Mid Semester	20%
End Semester	40%
Total	100%

1.11 Reference Textbooks

1. *Computer Networking: A Top-Down Approach* – James Kurose & Keith Ross
2. *Data Communications and Networking* – Behrouz A. Forouzan

QUICK RECAP

- Cloud $\neq$ magic $\rightarrow$ physical servers + networks.
- Cloud providers: AWS, Azure, GCP.
- Networks date to the 1960s (UCLA $\rightarrow$ SRI “lo” incident).
- Network scope: PAN $\rightarrow$ LAN $\rightarrow$ MAN $\rightarrow$ WAN.
- Internet = physical infrastructure, especially submarine cables (597 systems, 2025); threats = anchors/traulers, sharks.
- Real app usage (Instagram/Netflix) is far more complex than “WiFi $\rightarrow$ App.”
- Course = Top-Down Approach: Application $\rightarrow$ Transport $\rightarrow$ Network $\rightarrow$ Link $\rightarrow$ Physical.
- GPU clusters rely on ultra-fast networking (e.g., 200 Gbps InfiniBand).

2 Lecture 2: Moving Data Through the Core

EXAM FOCUS

Syllabus core areas: Packet vs. circuit switching; store-and-forward; delay components; L/R and d/s ; throughput; bottleneck; packet loss. **Queuing delay math is out of scope** – only transmission (L/R) and propagation (d/s) calculations are examinable.

2.1 Circuit Switching vs. Packet Switching

The lecture motivates this with “A Tale of Two Phone Calls.”

ANALOGY

Landline call: If Grandma pauses for 10 seconds, the dedicated circuit capacity remains reserved but idle – wasted.

WhatsApp (packet-based) call: If you pause for 10 seconds, no packets are sent, and that network capacity becomes available for someone else to use.

2.1.1 Circuit Switching – “The Road Not Taken”

- Classic telephone model: before communication starts, the network establishes a **dedicated end-to-end path**.
- The path is held **exclusively** for the entire duration of the call.
- **Advantage – Guaranteed:** once established, reserved capacity is yours; communication is predictable and stable.
- **Disadvantage – Wasteful for bursty data:** if you stop sending data, the reserved capacity stays idle and **cannot** be used by others.

2.1.2 Packet Switching

- Data (photo, webpage, voice clip) is divided into small independent chunks called **packets**.
- Each packet = **Header** (addressing information) + **Payload** (actual data).
- Packets can take **different routes** to the same destination.
- The network core forwards packets independently, **hop-by-hop**, router to router.
- **No built-in guarantees:** packets can be delayed in queues or dropped when a router buffer is full.

FORMULA

Store-and-Forward Rule: A router must *fully receive* the entire packet → check it → then forward the entire packet. A router does **not** need to know the complete end-to-end path – only the **next hop**.

2.2 Circuit Switching vs. Packet Switching – Comparison Table

Circuit Switching	Packet Switching
Dedicated path	Shared network
Resources reserved	Resources used on demand
Predictable	Best effort
Can waste capacity	More efficient
Traditional telephone model	Modern data networks
Poor for bursty traffic	Good for bursty traffic

2.3 Statistical Multiplexing

Unlike circuit switching, packet switching has **no reservations** – links are used only when packets actually need them. Two major advantages:

1. **Efficiency:** Real traffic is bursty; users don't transmit continuously. Multiple users can share the same link because it is statistically unlikely everyone bursts simultaneously.
2. **Resilience:** Packets are not tied to one fixed path. If a link or router fails, traffic can be rerouted – nothing was permanently reserved on the failed path.

EXAM FOCUS

Packet switching is described as “**The Seed of Everything TCP Does.**” Because the network core is best-effort/unreliable, TCP must later handle: detecting packet loss, retransmitting data, and pacing data – to make the network *feel* reliable to applications.

2.4 Sources of Delay – The Toll-Booth Caravan Analogy

ANALOGY

Imagine a caravan of cars passing through a **Toll Booth** onto a **Highway**. Each car passes the booth one at a time, taking a fixed time each (this represents **transmission delay**); after passing, cars travel together at the speed limit along the highway (this represents **propagation delay**).

Every packet experiences **four types of delay**:

#	Delay	Meaning	Formula
1	Processing Delay	Router examines header, checks bit errors (tiny, $\sim \mu s$)	–
2	Queuing Delay	Time waiting in a buffer behind other packets (variable)	–
3	Transmission Delay	Time to push <i>all</i> packet bits onto the link	L/R
4	Propagation Delay	Time for a bit to physically travel through the link	d/s

FORMULA

$$d_{\text{nodal}} = d_{\text{processing}} + d_{\text{queuing}} + d_{\text{transmission}} + d_{\text{propagation}}$$

$$d_{\text{transmission}} = \frac{L}{R} \quad \text{where } L = \text{packet length (bits)}, R = \text{link rate (bits/sec)}$$

$$d_{\text{propagation}} = \frac{d}{s} \quad \text{where } d = \text{distance (m)}, s = \text{signal speed (m/s)}$$

2.5 Transmission vs. Propagation Delay – Toll-Booth Mapping

Transmission → Toll Booth	Propagation → Highway
Time for every car to pass through the booth	Time for cars to travel down the highway
Depends on: packet size, link rate	Depends on: distance, signal speed
Formula: L/R	Formula: d/s
Higher link rate → lower transmission delay; more booths → faster	Does not depend on number of cars/packet size; more lanes $\neq$ faster propagation

EXAM FOCUS

Critical distinction: Transmission delay depends on packet size and link rate (L/R). Propagation delay depends **only** on distance and signal speed (d/s) – it is **independent** of packet size and bandwidth. This is one of the most tested conceptual distinctions in the course.

2.6 Worked Example: Transmission Delay

FORMULA

Given: $L = 8,000$ bits, $R = 2$ Mbps.

$$d_{\text{transmission}} = \frac{8000}{2,000,000} = 0.004\text{s} = \boxed{4 \text{ ms}}$$

Link Rate	Transmission Delay
2 Mbps	4 ms
4 Mbps	2 ms
8 Mbps	1 ms

Rule: Doubling the link rate → halves the transmission delay (inverse relationship).

2.7 Worked Example: Propagation Delay – The Satellite Case

For fiber, signal speed $s \approx 2 \times 10^8$ m/s.

EXAM FOCUS

The “500 Mbps link that still feels slow” scenario: A geostationary satellite orbits at $\approx 35,786$ km above the equator. A signal must travel Earth → Satellite → Earth, i.e., $\approx 71,500$ km, producing $\approx \boxed{240 \text{ ms round-trip}}$ delay from physics alone – **even though** the link is advertised at 500 Mbps (high bandwidth).

Key lesson: High bandwidth $\neq$ low latency. A connection can have huge bandwidth while still having significant propagation delay because propagation depends on distance, not bandwidth.

2.8 Throughput and Bottlenecks

Throughput = the rate at which data is **actually** received (bits/sec). It is **not** the advertised rate and **not** merely the theoretical rate – it is what actually reaches the destination.

FORMULA

End-to-End Throughput $\approx \min(\text{link rates along the path})$

Example: Path with links 10 Mbps $\rightarrow$ 2 Mbps $\rightarrow$ 5 Mbps.

Bottleneck = 2 Mbps $\Rightarrow$ End-to-End Throughput $\approx$ **2 Mbps** (regardless of the faster links).

EXAM FOCUS

“I pay for 300 Mbps – why is it buffering?” scenario: Your ISP plan (300 Mbps) is only the *access link* – just one hop on the full path (Netflix Server $\rightarrow$ Congested Link $\rightarrow$ Your ISP $\rightarrow$ Home WiFi $\rightarrow$ Your TV). The actual bottleneck could be a congested inter-network link, an overloaded server, home WiFi, or wall/interference issues. **Main lesson:** always ask “which hop is the slowest?”

2.9 Bridge to Lecture 3

The network core moves packets, but different networks/applications must agree on a common format and order of operations – this is the job of **protocol layers**, covered next.

QUICK RECAP

- Circuit switching = reserved, guaranteed, wasteful for bursty data.
- Packet switching = shared, best-effort, efficient + resilient (statistical multiplexing).
- Packet = Header + Payload; Store-and-Forward = receive full packet $\rightarrow$ check $\rightarrow$ forward.
- 4 delays: Processing, Queuing, Transmission (L/R), Propagation (d/s).
- Transmission depends on packet size/link rate; Propagation depends on distance/speed only.
- Throughput $\approx$ bottleneck (minimum) link rate on the path.
- High bandwidth $\neq$ low latency (satellite example: 240 ms RTT despite 500 Mbps).

3 Lecture 3: OSI and TCP/IP Models (Protocol Layers)

EXAM FOCUS

Syllabus core areas: OSI vs. TCP/IP; purpose of layering; protocols and services; PDUs; encapsulation/decapsulation; device-to-layer mapping.

3.1 Why Layering?

A network is too complicated to design, build, or debug as one single thing. Layering makes it manageable:

- Each layer performs **one specific job**.
- A layer communicates **only** with the layers directly above and below it.
- Each layer deliberately **knows nothing** about the rest of the system.

3.1.1 Recap Bridge from Lecture 2

Concept	Meaning
Packet > Circuit Switching	Data split into chunks that dynamically share links – efficient for bursty traffic
Store-and-Forward	Routers fully receive, verify, buffer a packet before forwarding
Anatomy of Delay	Every packet: processing, queuing, transmission, propagation delay
Throughput & Bottlenecks	End-to-end speed limited by the slowest bottleneck link

Core question: How can thousands of different networks, devices, OSs, applications, and vendors all agree on packet format, order of operations, and communication rules? **Answer: Layering and protocols.**

3.2 The Letter Analogy

ANALOGY

You write a letter, put it in an envelope, and give it to the postal service. The mail carrier does **not** care about the letter's contents – only the address on the envelope. Similarly, you don't need to know sorting offices, delivery vans, or flight routes. Each component does its own job. **If the carrier reads the letter** – the separation between layers breaks, privacy is lost, and the carrier must understand every possible message type. This introduces the idea behind **encryption** and controlling which devices can inspect traffic and how deeply.

3.3 What Layering Gives Us

1. **Each layer solves one problem** – can be understood independently.
2. **Layers can be replaced** – e.g., swap WiFi for Ethernet without rewriting everything above.
3. **Interoperability** – different vendors implement different layers and still communicate via agreed interfaces.
4. **Easier troubleshooting** – identify the problematic layer instead of investigating the whole system.

3.4 Encapsulation and Decapsulation

ANALOGY

Nesting-doll analogy (Sender): Data moves down the protocol stack. Each layer receives data from above → adds its own header → passes it downward.

Receiver: the reverse happens – receive → remove header → pass payload upward. This reverse process is called **decapsulation**.

Gift-wrapping analogy: Gift → Paper → Box → Ribbon. When sending: wrap in layers. When receiving: unwrap in reverse order. You cannot remove the inner wrapping before the outer wrapping.

EXAM FOCUS

Symmetry property: Layer N at the sender has a “private conversation” with Layer N at the receiver – the receiver’s Layer 3, for example, reads the header created by the sender’s Layer 3. Neither layer interferes with headers belonging to other layers. Encapsulation is what makes layering a *real mechanism*, not just a diagram.

3.5 The OSI Model

OSI = Open Systems Interconnection. Developed by **ISO (International Organization for Standardization)**. Divides network communication into **7 layers** to simplify network design and reasoning.

Why learn it?

- Everything you use (WhatsApp, games, Netflix) runs on it.
- Provides “one map” instead of treating “send data across the world” as one giant problem.
- Industry’s shared language (e.g., “That’s a Layer 4 issue” = Transport layer problem).
- Foundation for cybersecurity, cloud, AI infrastructure, backend engineering.

3.5.1 OSI Memory Tricks

Bottom-up (L1 → L7) Top-down (L7 → L1)

“Please Do Not Throw Sausage Pizza Away”

“All People Seem To Need Data Processing”

Layer #	Bottom-Up Word	Top-Down Word
1 – Physical	Please	Processing
2 – Data Link	Do	Data
3 – Network	Not	Need
4 – Transport	Throw	To
5 – Session	Sausage	Seem
6 – Presentation	Pizza	People
7 – Application	Away	All

3.6 The Seven OSI Layers – Full Detail

3.6.1 Layer 7 – Application

Job: Provides network services directly to end users/applications (web browsers, email, file transfer).

Unit of concern: Data.

ANALOGY

Waiter in a restaurant. The waiter takes your order (user request), relays it to the kitchen (network), and brings the food back (data). The customer never enters the kitchen; the chef never talks directly to the customer.

3.6.2 Layer 6 – Presentation

Job: Formats, encrypts, compresses data. Two headline responsibilities: **Translation + Encryption**.

- ASCII ↔ Unicode / character encoding conversions
- Compression/decompression (ZIP/unzip)
- Encryption/decryption
- Image encoding: PNG, JPEG, GIF

Analogy: Translator – converts, formats, and secures messages so sender/receiver understand each other.

3.6.3 Layer 5 – Session

Job: Establishes, manages, terminates communication sessions. Two named responsibilities: **Dialog control** and **Synchronization**. **Analogy:** Courtroom judge/panel moderator – starts the conversation, keeps it running, controls whose turn it is, ends the conversation.

3.6.4 Layer 4 – Transport

Job: Reliable, efficient delivery between two devices. Data is encapsulated with source/destination addressing, broken into pieces, and reassembled at the receiver. **Analogy:** Many conversations sharing one phone line – **multiplexing** ensures messages reach the correct recipient.

EXAM FOCUS

TCP lives at Layer 4. TCP handles loss detection, retransmission, and pacing – directly connecting to Lecture 2’s “best-effort, unreliable core” concept.

3.6.5 Layer 3 – Network

Job: Logical addressing, forwarding packets, communication across different networks. Objective: get packets from point A to point B. **Analogy:** GPS – finds routes for delivery trucks.

EXAM FOCUS

Layer 3 does **not** care whether the next network uses Ethernet, WiFi, or Fibre – this is what allows multiple different networks to form a “network of networks.”

3.6.6 Layer 2 – Data Link

Job: Reliable data transfer between two **directly connected** nodes (**one hop** scope). Two major responsibilities:

1. **Media Access Control (“One at a time!”):** decides who gets to transmit when multiple devices share a medium – prevents simultaneous transmission/interference.
2. **Error Checking:** verifies bits received = bits sent.

Also handles **MAC addressing**.

3.6.7 Layer 1 – Physical

Job: Transmits raw bits (0s and 1s) over a physical medium (coaxial cable, fibre optic, radio waves).

Analogy: Roads, wires, signals – no addresses, no routes, no meaning; just the physical mechanism for moving something.

EXAM FOCUS

Propagation delay belongs here. The speed of light in fibre is a Layer 1 fact – higher-level protocols cannot eliminate this physical limitation. Direct link to Lecture 2's d/s formula.

3.7 Layer 2 vs. Layer 3 – The Most Confused Distinction

EXAM FOCUS

Layer 3 = the whole journey (e.g., Computer A → Network 1 → Network 2 → Network 3 → Computer B).

Layer 2 = one direct link / one hop (e.g., A → B directly).

This is explicitly flagged in the lecture as one of the **most commonly confused** distinctions – high MCQ probability.

3.8 Seven OSI Layers at a Glance

Layer	Name	Main Job	Analogy
7	Application	Network services to users/apps	Waiter
6	Presentation	Translation, compression, encryption	Translator
5	Session	Starts, maintains, ends conversation	Judge/moderator
4	Transport	Reliable delivery, multiplexing	Many calls, one phone line
3	Network	Logical addressing and routing	GPS
2	Data Link	One-link delivery, MAC, error checking	One at a time
1	Physical	Moves raw bits through medium	Roads and wires

3.9 The TCP/IP Model

The TCP/IP model is a layered framework defining how data is transmitted over the Internet using protocols such as TCP and IP.

EXAM FOCUS

OSI = model used to think and teach.

TCP/IP = model the Internet actually runs on.

They are **not competitors** – OSI is a clean conceptual model; TCP/IP is the working system that evolved into the Internet's actual architecture.

3.10 OSI vs. TCP/IP – Comparison Table

OSI Model	TCP/IP Model
Open Systems Interconnection	Transmission Control Protocol / Internet Protocol
7 layers	4 layers
Developed by ISO	Developed by DARPA
Conceptual model	Used in actual data transmission between computers

QUICK RECAP

- Layering: each layer solves one problem, layers are replaceable, enables interoperability + easier troubleshooting.
- Encapsulation (sender: add headers going down) vs. Decapsulation (receiver: remove headers going up); symmetry between matching layers.
- OSI = 7 layers (ISO); TCP/IP = 4 layers (DARPA); OSI is conceptual, TCP/IP is what the Internet runs on.
- Layer jobs: 7-Application(data/services), 6-Presentation(translate/encrypt/compress), 5-Session(dialog control), 4-Transport(TCP, reliable delivery, multiplexing), 3-Network(IP, routing, whole journey), 2-Data Link(MAC, one hop, error check), 1-Physical(raw bits, propagation delay).
- **Layer 3 = whole journey; Layer 2 = one hop** – top exam trap.

4 Lecture 4: Networking Devices, Topologies & the Cloud Network

EXAM FOCUS

Syllabus core areas: Hub/switch/router/firewall/load balancer; star/mesh/hybrid; Regions, AZs, VPC, Edge Locations; physical-to-cloud mapping.

4.1 Why Do We Need Network Devices?

Key idea: network devices operate at different OSI layers and differ mainly in *how deeply* they inspect packets.

4.1.1 The Scaling Problem

If every computer directly connects to every other computer (full mesh, no central device), the number of cables required is:

FORMULA

$$\text{Cables required} = \frac{n(n-1)}{2}$$

Devices	Calculation	Cables
3	$3 \times 2/2$	3
10	$10 \times 9/2$	45
100	$100 \times 99/2$	4,950
1,000	$1000 \times 999/2$	499,500

EXAM FOCUS

Number of devices grows **linearly**, but number of cables grows **quadratically** ($\sim n^2$). Rule of thumb: **10× devices $\approx$ 100× cabling**. This is impractical for offices/data centers.

4.1.2 The Solution – The Pivot

Instead of Everyone $\leftrightarrow$ Everyone, use Everyone $\rightarrow$ **ONE central device**.

$$n \text{ devices need only } n \text{ cables (not } \frac{n(n-1)}{2} \text{)}$$

This is the fundamental reason networking hardware (hubs, switches, routers) exists.

4.2 Core Networking Devices

Devices introduced: Hub, Switch, Wireless router, Access point, Firewall, Load balancer, Router.

4.2.1 Hub – The 1980s Answer (Layer 1)

- Broadcasts incoming data to **all** connected devices.
- Does **not** understand MAC addresses.
- Operates at **Layer 1 – Physical**.
- Simply receives an electrical signal on one port and repeats it through all other ports.

Main problem: unnecessary traffic. A message for Computer A is sent to A+B+C+D...; every machine receives it, inspects it, realizes it isn't for them, and discards it – wasting the shared medium and increasing congestion as devices are added.

4.2.2 Switch – The 1990s Answer (Layer 2)

- Operates at **Layer 2 – Data Link**.
- Connects devices within a LAN using **MAC addresses**.
- Sends data to the correct device/port.
- Does **not** understand Layer 3/IP addresses.

ANALOGY

“A switch is like a hub with memory.” It learns MAC address → Port mappings, so instead of broadcasting every frame everywhere, it forwards only where needed.

4.2.3 Hub vs. Switch – Comparison Table

Feature	Hub	Switch
Era	1980s	1990s onward
Layer	L1 – Physical	L2 – Data Link
Addresses	None	MAC addresses
Forwarding	Broadcasts to every port	Sends to correct port
Main weakness	Unnecessary traf- fic/congestion	Cannot see IP/reach another network

4.2.4 Router – Reaching Another Network (Layer 3)

A switch cannot reach another network because it doesn't understand Layer 3. A router:

- Routes data between different networks.
- Operates at **Layer 3 – Network**.
- Understands IP addresses; uses logical addressing and routing.

Analogy: The router is like a GPS – it knows how to get packets from point A to point B across different networks.

4.2.5 Firewall

Distinction: **Router** asks “Where should this packet go?”; **Firewall** asks “Should this packet be allowed to enter?”

- A network security device that monitors traffic entering/leaving a network.
- Protects against unauthorized access and threats; only authorized traffic is allowed through.

Basic packet filtering inspects:

Packet Part	Firewall Checks
IP header	Source & destination IP address
TCP/UDP header	Source & destination port
Data	Not examined by simple packet filtering

EXAM FOCUS

Basic packet filtering considers **4 values**: Source IP, Destination IP, Source Port, Destination Port.

IP address → Layer 3; Port number → Layer 4. **A basic packet-filtering firewall reads two layers deep (L3 + L4).**

4.2.6 Load Balancer

Distributes incoming network traffic across multiple servers to **prevent one server from becoming overloaded**. Example given: 6 incoming requests distributed across 3 servers (instead of all 6 going to 1 server). Companies mentioned as relying on this: Netflix, Google, Amazon, Facebook.

EXAM FOCUS

Connection to Lecture 2's bottleneck concept: if the bottleneck is a **server** (not a network link), a load balancer helps by distributing traffic. Think: "Stop sending everything to one server" rather than "make one server faster."

4.3 Devices by OSI Layer – The Single Most Important Table

Device	Layer	What It Decides On
Hub	L1 – Physical	Nothing; repeats signals to every port
Switch	L2 – Data Link	MAC addresses within one LAN
Router	L3 – Network	IP addresses between different networks
Firewall	L3/L4 (+higher types)	IP addresses and port numbers; advanced firewalls inspect more
Load Balancer	L4/L7	Which server should receive the request
Wireless Router / Access Point	L1–L3 (role-dependent)	Provides wireless network access

EXAM FOCUS

Easy recall chain: Hub → Signal — Switch → MAC — Router → IP — Firewall → IP + Port — Load Balancer → Server

4.4 Network Topologies

Definition: the arrangement of devices and their connections within a network, including how data flows through it. Types covered in detail: **Star**, **Mesh**, **Hybrid** (Ring and Bus are listed as topology types but not detailed further).

4.4.1 Star Topology

Every device connects to a **central hub or switch**.

- Centralized structure; data goes through the central device.
- Easier diagnostics and maintenance; popular for LANs; scales relatively well.
- **Main weakness:** the central device is a **single point of failure** – if it fails, all connected devices lose connectivity.

4.4.2 Mesh Topology

Every device is directly connected to every other device.

- **Advantages:** multiple possible paths, high reliability; if one path fails, another can be used.
- **Disadvantage:** huge number of connections ($n(n-1)/2$ – same scaling problem as before). Nicknamed "**Spaghetti networks**."
- **Wireless mesh:** still useful without physical cabling – extends WiFi coverage since adding nodes creates additional paths.

4.4.3 Hybrid Topology

Combines two or more topology types (e.g., Star + Ring + Bus).

- Flexible; can combine benefits of multiple topologies; suitable for large/complex networks.
- Real-world networks are often hybrids because different organizational parts have different requirements.

4.4.4 Choosing a Topology by Environment

Environment	Topology	Reason	Example Devices
Small office	Star	Easy setup, centralized control	Laptops, printers, routers
Smart home	Mesh	Reliable direct device-to-device comms	Smart bulbs, cameras
Data centre	Hybrid	Combines benefits of multiple topologies	Servers, storage units

4.4.5 Fault Tolerance & Scalability

Topology	Fault Tolerance	Scalability
Star	Medium	Good
Mesh	High	Excellent
Hybrid	High	Hard

EXAM FOCUS

There is no single “best” topology. The correct exam framing is: “Best for what, at what size, and at what cost?”

4.5 Cloud Networking – Hardware Meets Software

Traditional	Cloud Era
Physical box	Virtual instance
Copper cables	Virtualized overlays
Manual wiring	API calls / console clicks

EXAM FOCUS

Cloud networking does not make prior concepts obsolete – a load balancer in AWS is still a load balancer; a subnet still partitions a Layer 3 network. The cloud changes *how you instantiate* the concepts, not the concepts themselves.

4.5.1 AWS Foundational Concepts

- **Region:** an isolated geographic area (example: Mumbai) containing multiple data-center clusters. The lecture states there are **30 or more regions globally**, connected by an ultra-low-latency backbone.
- **Edge Location:** CDN endpoints that cache content physically closer to users, to reduce latency. Direct link to Lecture 2: since $d_{\text{propagation}} = d/s$, moving content physically closer decreases distance d , which decreases propagation delay. (Also linked to Netflix placing appliances inside ISP networks.)

- **Availability Zone (AZ):** one or more discrete data centers with redundant power, networking, and connectivity – acts as an **isolated fault domain**. Deploying across multiple AZs lets an application survive localized failures (fire, flood, power-grid failure) – if one AZ fails, the application can continue using another.
- **VPC (Virtual Private Cloud):** your own private, software-defined network inside the AWS cloud. You control IP address ranges, subnets, and routing.
- **Subnets:** map to specific AZs; partition the Layer 3 network; help with scaling and security. VPC addressing uses **CIDR** notation (detailed CIDR math is covered in a later lecture on IP addressing, not this one) – for now: you receive a block of IP addresses and decide how to divide it into smaller networks.

QUICK RECAP

- Direct connections don't scale: $n(n-1)/2$ cables vs. centralized: n cables.
- Hub(L1, broadcast, no addressing) → Switch(L2, MAC, learns port mapping) → Router(L3, IP, cross-network).
- Firewall = “should it be allowed?” – checks Source/Dest IP + Source/Dest Port (L3+L4).
- Load Balancer = “which server should handle this?” – solves server-side bottlenecks.
- Star(central point of failure, good for offices), Mesh($n(n-1)/2$ links, highest fault tolerance, good for smart homes), Hybrid(flexible, common in data centers).
- AWS: Region(geo area, 30+) → AZ(isolated fault domain) → VPC(private software-defined network) → Subnet(maps to AZ) → Edge Location(reduces latency via proximity).

5 Lecture 5: HTTP, P2P & REST (Application Layer)

EXAM FOCUS

Syllabus core areas: Client-server vs. P2P; HTTP request/response; methods/status codes; HTTP/1.1 vs. 2 vs. 3; HoL blocking; REST and CRUD.

5.1 Network Architectures

5.1.1 Client-Server Architecture

A **centralized** architecture where powerful servers store data, process requests, and serve multiple dependent clients. Multiple computers can request services/data from a central server.

5.1.2 Peer-to-Peer (P2P) Architecture

A **decentralized** architecture where all devices (peers) can act as both clients and servers. All peers have equal privileges; there is **no single central server** responsible for everything.

EXAM FOCUS

Client-Server = **Centralized** (servers provide services to clients).

P2P = **Decentralized** (peers act as both clients and servers, equal privileges).

5.2 How Systems Communicate

Basic interaction: Client → Request → Server; Client ← Response ← Server. This is the gateway concept into the Application Layer.

5.3 The Application Layer (Layer 7)

The direct interface between software programs and the underlying network (e.g., web browsers, email clients). It interprets user commands and enables data exchange across systems.

Application Layer protocols mentioned: HTTP, HTTPS, FTP, SMTP, Telnet, DNS, POP3, IRC, NNTP, DHCP, SNMP – supporting file transfer, web surfing, email, and virtual terminals.

5.4 HTTP – The Web’s Language

HTTP = Hypertext Transfer Protocol, defined as a **stateless request-response protocol**.

Basic process: Client (browser) sends a **REQUEST** → Server processes it → Client receives a **RESPONSE**. Each exchange is independent – the server does not automatically remember previous requests.

5.4.1 HTTP Request Structure

1. Request Line
2. Headers
3. Empty Line
4. Body (optional)

Example:

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Chrome
Accept-Language: en-us
```

The body is generally relevant for methods such as POST and PUT (and PATCH).

5.4.2 Request Line – Three Parts

1. **Method:** the operation the client wants performed (GET, POST, PUT, DELETE, ...)
2. **URL:** the requested resource/endpoint (e.g., `/index.html`, `/api/data`)
3. **HTTP Version:** e.g., HTTP/1.1, HTTP/2

Example: GET `/index.html` HTTP/1.1

5.4.3 HTTP Headers (Request)

- **User-Agent:** information about client software (e.g., Chrome)
- **Host:** domain name of the server (e.g., `example.com`)
- **Content-Type:** type of data being sent (`application/json`, `text/html`)
- **Accept:** types of data the client can process

5.4.4 HTTP Response Structure

1. Status Line
2. Headers
3. Empty Line
4. Body

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 88
Server: Apache/2.4.2
```

5.4.5 HTTP Status Line – Three Parts

1. **HTTP Version:** e.g., HTTP/1.1
2. **Status Code:** three-digit result (200, 404, 500)
3. **Status Message:** short description (OK, Not Found, Internal Server Error)

Example: HTTP/1.1 200 OK

5.4.6 HTTP Response Headers

- **Content-Type:** type of data in response
- **Content-Length:** length of response body in bytes
- **Server:** information about server software
- **Date:** date/time response was sent

5.5 HTTP Status Codes

Code	Meaning
200 OK	Request successful
301 Moved Permanently	Resource permanently moved to a new URL
302 Found	Temporary redirection
403 Forbidden	Server refuses authorization
404 Not Found	Requested resource cannot be found
500 Internal Server Error	Server encountered an internal error

EXAM FOCUS

Status code categories (by first digit):

1xx → Informational 2xx → Success 3xx → Redirection 4xx → Client error 5xx → Server error

5.6 Statelessness and Cookies

EXAM FOCUS

If HTTP is stateless, how does a shopping cart work? Answer: **Cookies & Session Tokens** – they allow applications to maintain information across otherwise independent HTTP requests.

5.7 Evolution of HTTP

The progression: **HTTP/1.0** → **HTTP/1.1** → **HTTP/2** → **HTTP/3**. The lecture uses an online-shopping delivery analogy: ordering a shirt, pants, shoes, socks $\approx$ a browser requesting HTML, CSS, JavaScript, Images. The question driving the evolution: *how should these items be delivered efficiently?*

5.7.1 HTTP/1.0 – 1996

ANALOGY

Compared to a truck that carries **one item per trip**: “I deliver one file at a time.”

- One request at a time.
- Every file requires a **new connection**.
- No persistent TCP connection.
- Slow, repetitive, inefficient.

5.7.2 HTTP/1.1 – 1997

- Key features: **Keep-Alive connections**, caching, Host headers.
- Analogy: “One truck arrives with all four items,” but still unloads them one at a time.
- Major improvement: one TCP connection can remain open (instead of a new connection per file).
- **Major remaining problem**: Head-of-Line (HoL) blocking.

5.7.3 Head-of-Line (HoL) Blocking

With HTTP/1.1, one connection means responses must arrive in order (**FIFO – First In, First Out**). Example: Slow Image → CSS → JS → Icon – if the image is slow, the others are blocked even if already ready. **One slow response can delay everything behind it.**

5.7.4 HTTP/1.1 Pipelining

HTTP/1.1 optionally supports pipelining: multiple requests can be sent without waiting for previous responses, but **responses must still arrive in request order (FIFO)**. Only **safe methods** can be pipelined: GET, HEAD, OPTIONS. POST/PUT are excluded because they may have side effects and alter destination state. **Even with pipelining, HoL blocking remains.**

5.7.5 HTTP/2 – 2015

ANALOGY

Restaurant tray analogy: instead of serving each item one at a time, **serve the entire tray** and let the customer choose.

Features:

1. **Single TCP Connection.**
2. **Binary Protocol** (instead of text-based HTTP) – referred to as the framing layer.
3. **Multiplexing:** multiple requests/responses simultaneously over one connection.
4. **Server Push:** server can proactively send resources without an explicit request.

5.7.6 HTTP/3 + QUIC – June 2022

Built on **QUIC**: “a UDP-based transport protocol developed by Google.”

HTTP/2 stack	HTTP/3 stack
HTTP/2	HTTP/3
TCP	QUIC
IP	UDP
	IP

Features:

1. Latest HTTP version (as presented in the lecture).
2. Uses QUIC instead of TCP.
3. Multiplexed streams without blocking.
4. Designed to solve HoL blocking – loss affecting one stream does **not** necessarily block other streams.

5.8 HTTP/1.1 vs. HTTP/2 vs. HTTP/3 – Master Comparison

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC/UDP
Connections	Multiple/persistent	One TCP connection	QUIC
Multiplexing	Limited/pipelining	Yes	Yes
Encoding	Text	Binary	Binary/QUIC-based
HoL blocking	Yes	TCP-level still possible	Stream-level loss isolation
Main improvement	Keep-alive	Multiplexing	Independent streams over QUIC

5.8.1 The Head-of-Line Blocking Evolution – Deeper View

- **HTTP/1.1:** multiple separate TCP connections simulate parallelism → problem: costly setup + overhead.
- **HTTP/2:** many streams multiplexed over one TCP connection → **application-level HoL is fixed**, but **TCP-level HoL remains** – if one TCP segment is lost, **all streams can stall**.
- **HTTP/3:** QUIC over UDP → streams are truly independent; if one stream experiences loss, **only that stream waits** – others continue.

EXAM FOCUS

Highest-yield HoL fact: HTTP/2 fixes *application-level* HoL blocking via multiplexing but **still suffers TCP-level HoL blocking**. Only HTTP/3 (via QUIC/UDP) achieves true stream-level independence.

5.9 REST APIs

After HTTP, the lecture moves to Application ↔ Application communication: *what happens when one application needs to communicate with another?* → **REST APIs**.

REST API sits between: Clients → REST API → Database. Clients make HTTP requests (GET, POST, PUT, PATCH, DELETE); the REST API performs the appropriate operation, interacts with the database, and returns a response (JSON or XML).

5.9.1 HTTP Methods ↔ CRUD Mapping

Method	Operation
GET	Read / retrieve data
POST	Create
PUT	Update / replace
PATCH	Update / modify (partial)
DELETE	Delete

5.9.2 REST Request/Response Flow

Client $\xrightarrow{\text{HTTP Request}}$ REST API $\xrightarrow{\text{Query/update}}$ Database $\xrightarrow{\text{Result}}$ REST API $\xrightarrow{\text{HTTP Response}}$ Client

5.10 OpenAPI

An **OpenAPI contract** sits between an **API Producer** (defines/publishes the API specification) and **API Consumers** (frontend developers, AI agents & LLMs, third-party developers, API testing tools). Reference example given: the Swagger Petstore.

QUICK RECAP

- Client-Server(centralized) vs. P2P(decentralized, equal privileges, no central server).
- HTTP = stateless request-response protocol, Layer 7.
- Request: Request Line + Headers + Empty Line + Body(optional). Response: Status Line + Headers + Empty Line + Body.
- Status codes: 1xx info, 2xx success, 3xx redirect, 4xx client error, 5xx server error.
- Statelessness solved by Cookies/Session Tokens.
- Evolution: 1.0(new connection per file) → 1.1(Keep-Alive, but HoL blocking) → 2(binary, multiplexed, server push, but TCP-level HoL remains) → 3(QUIC/UDP, true stream independence).
- REST maps HTTP methods to CRUD: GET=Read, POST=Create, PUT=Update/Replace, PATCH=Partial Update, DELETE=Delete.
- OpenAPI = API specification/contract between producer and consumers.

6 Lecture 6: HTTPS and the TLS Handshake

EXAM FOCUS

Syllabus core areas: Security goals; symmetric vs. asymmetric crypto; certificates/CAs; chain of trust; TLS handshake; MITM defence; HTTPS layering.

6.1 The Padlock Question

The browser padlock does **not** simply mean “this website is safe.” It indicates:

- ✗ Safe website? – **Not necessarily.**
- ✓ Encrypted connection? – Yes.
- ✓ Verified identity? – Yes.

EXAM FOCUS

HTTPS provides an encrypted connection and server identity verification, but the padlock does NOT guarantee the website itself is trustworthy. This is a classic conceptual MCQ trap.

6.2 Why HTTP Is Insecure

HTTP is plain text by design – no built-in security; vulnerable unless HTTPS is used. An attacker capable of capturing traffic may be able to understand the communication content between client and server.

6.3 HTTPS = HTTP + TLS

FORMULA

$$\text{HTTPS} = \text{HTTP} + \text{TLS}$$

Historically HTTPS used **SSL (Secure Socket Layer)**; modern HTTPS uses **TLS (Transport Layer Security)**. TLS encrypts data exchanged between client and server. TLS sits between the application protocol and the transport layer (in HTTP/2 and HTTP/3, TLS-related security sits above TCP/UDP respectively).

6.4 Cryptography Fundamentals

Definition: the practice of securing information by converting readable text into unreadable code (ciphertext) and back again.

Plaintext → Encryption → Ciphertext → Decryption → Plaintext

An encryption key is used during encryption; a decryption key is used during decryption.

6.4.1 Three Core Goals of Cryptography

Goal	Meaning
Confidentiality	Privacy – only authorized parties can access info. Simple meaning: <i>No eavesdropping</i>
Integrity	Data remains accurate, unaltered in transit/storage. Simple meaning: <i>No tampering</i>
Authentication	Verifies identities of sender/receiver. Simple meaning: <i>Verified identity</i>

EXAM FOCUS

Memorize triad: **Confidentiality** → No eavesdropping; **Integrity** → No tampering; **Authentication** → Verified identity.

6.5 Symmetric vs. Asymmetric Encryption

6.5.1 Symmetric Encryption

The **same secret key** is used for both encryption and decryption. Both parties need access to the same secret key.

EXAM FOCUS

Problem with symmetric encryption: *How do we share the key securely?* If two strangers have never communicated, they need the same secret key without an attacker intercepting it – this is the **key distribution problem**.

6.5.2 Asymmetric Encryption

Uses **two different keys**:

- **Public Key** – can be shared publicly.
- **Private Key** – must remain secret.

Flow: Sender encrypts plaintext using the recipient's **Public Key** → Ciphertext → Recipient decrypts using their **Private Key** → Plaintext.

6.6 The Key Insight of TLS

FORMULA

TLS uses **both** types of encryption:

Asymmetric Encryption → Share a secret safely (Secure setup)

Symmetric Encryption → Fast communication (efficient data transfer)

Why not use only symmetric? Because strangers cannot securely share a key – the initial key-sharing problem remains. TLS therefore combines asymmetric techniques (establishing trust/secrets) with symmetric encryption (efficient ongoing communication).

6.7 Trusting a Server: Digital Certificates

The lecture illustrates the risk of look-alike domains (e.g., a homoglyph domain vs. **microsoft.com**) – a familiar-looking domain is not proof of identity. The client needs a mechanism to verify “is this actually the server it claims to be?” → **digital certificates**.

6.7.1 What Is a Certificate?

A digital certificate provides information used to establish the identity of a website/server. Typical fields: Subject name, Common name, Issuer name, Country/Region, Organization, Serial number, Validity information. (Example used: a certificate for **canva.com** showing issuer and validity period.)

6.8 The TLS Handshake – Full Flow

FORMULA

Step 1 – ClientHello: Client sends supported TLS versions, cipher suites, and a random nonce.

Step 2 – ServerHello + Certificate: Server selects a cipher, sends its certificate, public key info, and CA signatures.

Step 3 – Certificate Validation: Client validates the certificate chain (via a Certificate Authority) to establish server identity and prevent **Man-in-the-Middle (MITM) attacks**. If valid → client trusts the server. If invalid → browser can flag the site as not secure.

Step 4 – Key Exchange: Client and Server derive a **shared session key** (this is where asymmetric → symmetric handoff occurs).

Step 5 – Encrypted Communication: Application data is encrypted/decrypted using the shared session key – the “encrypted tunnel.”

EXAM FOCUS

Full TLS handshake sequence for MCQ ordering questions:

ClientHello → ServerHello + Certificate → Certificate Validation → Key Exchange → Shared Session Key → E

6.9 Certificate Authority (CA) and Chain of Trust

A **Certificate Authority** validates/signs digital certificates.

Certificate → Certificate Authority → Client trusts certificate

This trust relationship forms the **chain of trust**.

6.10 Real-World Deployment: AWS

Lab objective: adding a certificate using **AWS Certificate Manager (ACM)** – moving from TLS theory to actual cloud deployment.

6.11 Symmetric vs. Asymmetric – Master Comparison

Symmetric	Asymmetric
Same secret key	Public + private keys
Fast	Used for secure establishment
Key must be shared securely	Public key can be distributed freely
Used for efficient/bulk communication	Solves key-sharing/trust problems

FORMULA

Asymmetric → Secure setup

Symmetric → Fast communication

QUICK RECAP

- Padlock icon = encrypted + identity-verified connection, **not** a trustworthiness guarantee.
- HTTPS = HTTP + TLS (modern; historically SSL).
- 3 cryptography goals: Confidentiality(no eavesdrop), Integrity(no tamper), Authentication(verified identity).
- Symmetric = 1 key, fast, hard to share safely. Asymmetric = public/private key pair, used for secure key establishment.

- TLS uses both: Asymmetric for secure setup, Symmetric for fast ongoing communication.
- Digital certificate = proof of server identity; validated via a Certificate Authority (chain of trust); prevents MITM attacks.
- TLS Handshake order: ClientHello → ServerHello+Certificate → Certificate Validation → Key Exchange → Shared Session Key → Encrypted Communication.
- AWS lab tool: ACM (AWS Certificate Manager).

7 Master Formula Sheet (Lecture 2 Scope)

ALL EXAMINABLE FORMULAS

$$\text{Cables (full mesh)} = \frac{n(n-1)}{2}$$

$$\text{Cables (centralized device)} = n$$

$$d_{\text{transmission}} = \frac{L}{R} \quad (L = \text{bits}, R = \text{bits/sec})$$

$$d_{\text{propagation}} = \frac{d}{s} \quad (d = \text{distance}, s \approx 2 \times 10^8 \text{ m/s in fiber})$$

$$d_{\text{nodal}} = d_{\text{processing}} + d_{\text{queuing}} + d_{\text{transmission}} + d_{\text{propagation}}$$

$$\text{Throughput}_{\text{end-to-end}} \approx \min(\text{link rates along the path}) \quad (\text{bottleneck rule})$$

Reminder: Queuing-delay *calculations* are NOT in scope – only conceptual understanding that it exists and is variable/buffer-dependent.

8 Rapid-Fire MCQ Recall Sheet

- PAN → LAN → MAN → WAN (scope order)
- UCLA → SRI, word “login,” only “lo” got through (1960s)
- 597 submarine cable systems (2025); threats = anchors/trawlers, sharks
- Course approach: Top-Down (Application → Physical)
- Circuit switching: dedicated, guaranteed, wasteful for bursty data
- Packet switching: shared, best-effort, statistical multiplexing
- Packet = Header + Payload
- Store-and-forward: full receive → check → full forward; router only needs next hop
- Transmission delay depends on L , R ; Propagation depends on d , s only
- High bandwidth ≠ low latency (satellite: 240 ms RTT)
- Throughput ≈ bottleneck (min) link rate
- Layering: each layer solves 1 job, replaceable, interoperable, easier debugging
- Encapsulation (sender, add headers) vs. Decapsulation (receiver, remove headers)
- OSI = 7 layers, ISO. TCP/IP = 4 layers, DARPA
- OSI = conceptual; TCP/IP = what Internet actually runs on
- L7 Application(waiter) / L6 Presentation(translator) / L5 Session(judge) / L4 Transport(phone line, TCP) / L3 Network(GPS, IP) / L2 Data Link(MAC, one hop) / L1 Physical(raw bits)
- Layer 3 = whole journey; Layer 2 = one hop
- Propagation delay is a Layer 1 fact
- $n(n-1)/2$ cables direct vs. n cables centralized
- Hub = L1, broadcast, no addressing
- Switch = L2, MAC address learning, “hub with memory”
- Router = L3, IP, cross-network, GPS analogy
- Firewall = L3/L4, checks Src/Dst IP + Src/Dst Port (4 values)
- Load Balancer = L4/L7, distributes traffic across servers
- Star = central device, single point of failure, good for small office
- Mesh = $n(n-1)/2$ links, “spaghetti networks,” highest fault tolerance, good for smart home
- Hybrid = combination, common in data centers, hard to scale
- AWS Region = geographic area (30+ globally)
- AZ = isolated fault domain, redundant power/network
- VPC = private software-defined network; Subnet maps to AZ
- Edge Location = CDN cache closer to user → lower propagation delay
- Client-Server = centralized; P2P = decentralized, equal peers
- HTTP = stateless request-response protocol, Layer 7
- Request: Request Line + Headers + Empty Line + Body
- Response: Status Line + Headers + Empty Line + Body
- Status codes: 1xx info, 2xx success, 3xx redirect, 4xx client error, 5xx server error

- 200 OK, 301 Moved Permanently, 302 Found, 403 Forbidden, 404 Not Found, 500 Internal Server Error
- Statelessness fixed by Cookies/Session Tokens
- HTTP/1.0 = new connection per file (1996)
- HTTP/1.1 = Keep-Alive, but HoL blocking (1997)
- Pipelining: only safe methods (GET/HEAD/OPTIONS); still FIFO; HoL remains
- HTTP/2 (2015) = binary, single TCP connection, multiplexing, server push; TCP-level HoL still possible
- HTTP/3 (2022) = QUIC over UDP, independent streams, solves HoL via loss isolation
- REST CRUD: GET=Read, POST=Create, PUT=Update/Replace, PATCH=Partial update, DELETE=Delete
- OpenAPI = contract between API producer and consumers
- Padlock = encrypted + identity-verified, NOT a safety guarantee
- HTTPS = HTTP + TLS (was SSL)
- Crypto goals: Confidentiality(no eavesdrop), Integrity(no tamper), Authentication(verified identity)
- Symmetric = 1 shared key, fast, key-distribution problem
- Asymmetric = public/private key pair, secure setup
- TLS: Asymmetric → secure setup; Symmetric → fast data transfer
- Certificate = proves server identity; validated via CA (chain of trust); prevents MITM
- TLS Handshake: ClientHello → ServerHello+Cert → Cert Validation → Key Exchange → Session Key → Encrypted Data
- AWS Certificate Manager (ACM) = tool for deploying TLS certs on AWS

9 Cross-Lecture Concept Map (Scenario-Based Prep)

EXAM FOCUS

The syllabus explicitly calls out **scenario-based application** – expect MCQs that combine ideas across lectures. Use this map to connect them.

- **Netflix buffers at night (L1)** → explained via **bottleneck/throughput (L2)** → possibly a **server overload**, solved by a **load balancer (L4)** → delivered faster via **Edge Locations** reducing propagation delay (L4) → streamed over **HTTP/2 or HTTP/3** to avoid HoL blocking (L5) → secured via **HTTPS/TLS (L6)**.
- **500 Mbps satellite link still laggy (L2)** → propagation delay (d/s) is independent of bandwidth → same physical principle explains why **Edge Locations (L4)** help: reducing distance d reduces latency.
- **A packet crossing 3 networks (L3: Layer 3 = whole journey)** → physically passes through **routers (L4, Layer 3 devices)** at each hop, with **switches (L4, Layer 2)** handling delivery within each LAN, and possibly a **firewall (L4)** inspecting IP+port at network boundaries.
- **Shopping cart persists across pages (L5)** → HTTP is stateless → solved with cookies/session tokens → conceptually parallels **TLS session keys (L6)**, which also persist state (a session) across an otherwise per-message cryptographic exchange.
- **Encapsulation (L3)** is the general principle; **HTTP request/response structure (L5)** and **TLS record/handshake messages (L6)** are concrete instances of headers wrapping payloads at their respective layers.
- **AWS AZ failure tolerance (L4)** parallels **packet-switching resilience (L2)**: both rely on redundancy/alternate paths rather than a single fixed reservation.